Escaping the Fire

Choosing safe walking routes across a campus, with A*

Where we are. In the companion guide, *Finding a Fire with Cameras*, we used photographs from three cameras to work out exactly where a fire is — a six-cornered patch of ground about 0.27 hectares in size, centred at (426, 80) on the campus map.

The new question. People need to walk from where they are to the assembly point. There are many possible routes across campus. Some pass close to the fire; some do not. *Which routes should we tell people to take?*

The plan. Four steps:

Step 1 Turn the campus into a **graph**: dots for places, lines for walkways.

Step 2 Let the fire change the **cost** of each walkway. Near the fire, walking 100 m should "feel like" much more.

Step 3 Use **A*** (say "A-star") to find the cheapest route through that graph.

Step 4 Find a *few* good routes, not just one — because a single suggestion is fragile.

Step 5 Answer it for *everybody*, wherever they happen to be standing — from a single search.

A working C++ program, `campus_evacuation.cpp`, comes with this guide and produces every number in it.

What you need to know already. The distance formula $\sqrt{(x_2-x_1)^2+(y_2-y_1)^2}$, how to read coordinates off a map, and how to follow a list of steps. That is genuinely all — there is no trigonometry in this guide and no calculus.

Contents

1 Step 1 — The campus becomes a graph **2**
1.1 How long is each walkway? . . . 2

2 Step 2 — The fire changes the cost of walking **2**
2.1 How close does a walkway get to the fire? . . . 3
2.2 Turning a distance into a danger factor . . . 4

3 Step 3 — Finding the cheapest route with A* **5**
3.1 The obvious method, and why it is wasteful . . . 5
3.2 A*: Dijkstra with a sense of direction . . . 5
3.3 What makes a good hint? . . . 6
3.4 The algorithm, in words . . . 6

4 Step 4 — Suggesting a few routes, not just one **7**
4.1 How to get a second route . . . 7
4.2 Rejecting routes that are secretly the same . . . 7

5 The results **7**

6 Step 5 — When people start from everywhere **9**
6.1 The obvious method, and the better one . . . 9
6.2 Why the hint has to go . . . 9
6.3 What the search leaves behind: a signpost at every place . . . 9
6.4 The signpost for every building . . . 10
6.5 Two answers that look wrong, and are right . . . 11

7 The C++ program **12**
7.1 The three settings you are most likely to change . . . 12
7.2 Distance from a point to a walkway . . . 12
7.3 The danger factor . . . 12
7.4 A* itself . . . 13
7.5 Reading the route back . . . 13

A Every walkway, and what the fire did to it **13**

B Answers to "Try it yourself" **14**

C Words you now know **15**

D If you want to take this further **15**

1 Step 1 — The campus becomes a graph

A map has far too much detail. For route-finding, almost all of it can be thrown away. What matters is only:

- the **places** you can be — building entrances, path junctions, car parks;
- the **walkways** that join one place directly to another;
- how **long** each walkway is.

Draw a dot for each place and a line for each walkway, and you have a **graph**. Mathematicians call the dots *nodes* (or vertices) and the lines *edges*. A subway map is exactly this idea: it throws away the true shape of the tunnels and keeps only what connects to what.

Our campus has **16 places** and **25 walkways** (Figure 1). Everything is measured in metres on the same map as the fire, so the coordinates in this guide and in the camera guide mean the same thing.

1.1 How long is each walkway?

The length of a walkway is just the distance between the two places it joins — the distance formula:

$$\boxed{\text{length} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}} \tag{1}$$

Example. Dorm A is at $(80,\ 300)$ and the Library is at $(250,\ 330)$. So that walkway is

$$\sqrt{(250-80)^2 + (330-300)^2} = \sqrt{170^2 + 30^2} = \sqrt{28900 + 900} = \sqrt{29800} = 172.6 \text{ m}.$$

2 Step 2 — The fire changes the cost of walking

Here is the heart of the whole idea.

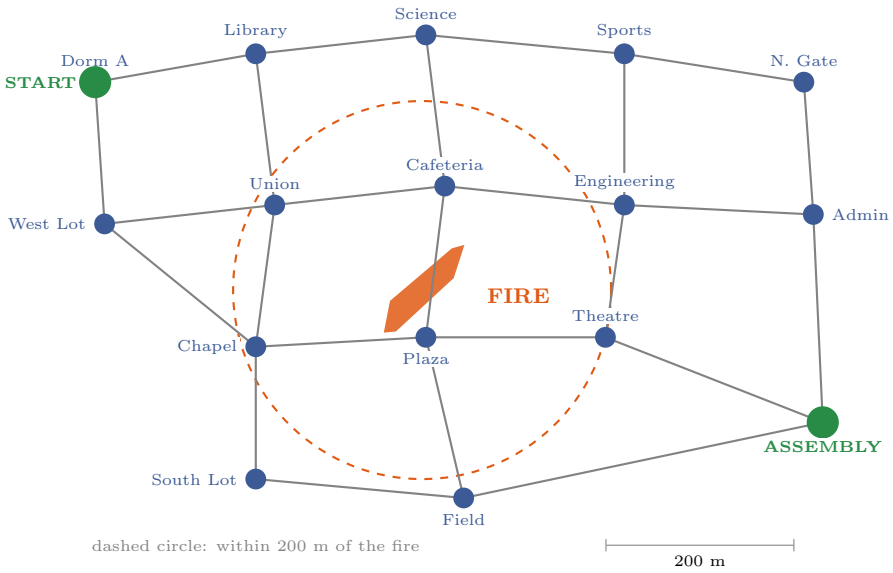


Figure 1: The campus as a graph. Blue dots are places; grey lines are walkways. The orange patch is the fire, exactly as the three cameras located it. Everyone starts at **Dorm A** (top left) and must reach the **Assembly Point** (bottom right).

If there were no fire, the best route would simply be the shortest one. But there *is* a fire, and a walkway that passes close to it is a bad choice even if it is short. We need a way to say "this walkway is short, but I would rather not."

The trick: make dangerous walkways *pretend* to be longer.
We give every walkway an **effective length**:

$$\text{effective length} \;=\; \text{real length} \;\times\; \text{danger factor}$$

The danger factor is 1 for a walkway far from the fire (no change at all), and grows as the walkway gets closer. A 100 m walkway with a danger factor of 6 counts as 600 m.

Then we simply look for the route with the smallest *total effective length*, and the route-finder automatically avoids the fire — without ever being told about fire. It just thinks those walkways are long.

2.1 How close does a walkway get to the fire?

To work out the danger factor we first need a distance: the **shortest gap** between the walkway (a line segment) and the fire (a six-sided patch).

The building block is the distance from a single *point* to a *segment*. Imagine walking along the segment from end A to end B and watching the point. Write your progress as a number t: at A you have $t = 0$, at B you have $t = 1$, halfway is $t = 0.5$. The closest spot is at

$$t = \frac{(P_x - A_x)(B_x - A_x) + (P_y - A_y)(B_y - A_y)}{(B_x - A_x)^2 + (B_y - A_y)^2}. \tag{2}$$

If t comes out less than 0 or more than 1, the closest spot is off the end of the segment, so we **clamp** it back to 0 or 1 — meaning the nearest point is simply whichever endpoint is closer. Then measure the distance to that spot with (1).

From points to whole shapes. The gap between two segments is the smallest of four point-to-segment distances: each of the four endpoints measured against the other segment.

And the gap between a walkway and the fire patch is the smallest gap to any of the fire's six sides — or exactly zero if the walkway runs into the patch.

2.2 Turning a distance into a danger factor

Now we convert "how far away" into "how much worse". We use a straight-line ramp with three simple rules and three numbers we choose in advance:

Name	Value	Meaning
blocking distance	25 m	closer than this, the walkway is **closed**
safe distance	200 m	further than this, the fire is ignored
worst danger	8×	the penalty just at the blocking edge

$$\text{danger} = \begin{cases} \text{CLOSED} & \text{if } d \le 25 \text{ m}, \\ 1 + 7 \times \dfrac{200 - d}{200 - 25} & \text{if } 25 < d < 200 \text{ m}, \\ 1 & \text{if } d \ge 200 \text{ m}. \end{cases} \tag{3}$$

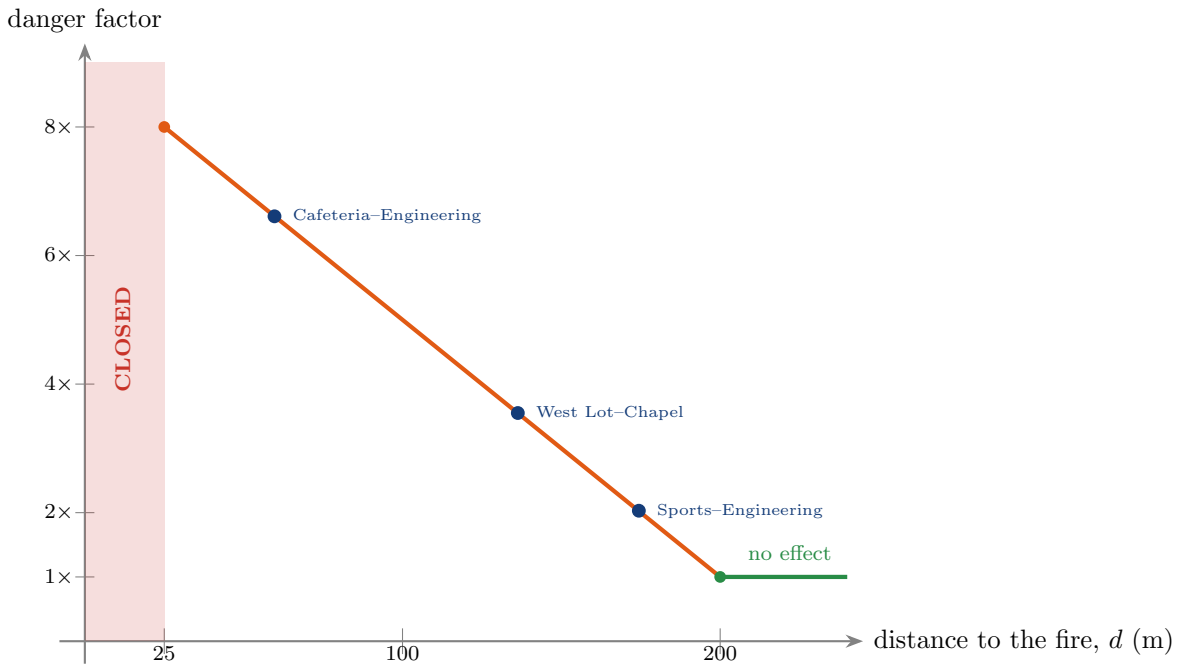


Figure 2: The danger factor as a straight-line ramp. Walkways closer than 25 m are closed outright; beyond 200 m the fire is ignored. Blue dots are three real walkways from our campus.

Example. The Cafeteria–Engineering walkway is 191.0 m long and its closest approach to the fire is $d = 59.7$ m. Since that is between 25 and 200:

$$\text{danger} = 1 + 7 \times \frac{200 - 59.7}{175} = 1 + 7 \times \frac{140.3}{175} = 1 + 7 \times 0.8017 = 6.61.$$

So its effective length is $191.0 \times 6.61 = \mathbf{1263.5}$ m. To the route-finder, that 191-metre walkway looks like more than a kilometre — and it will go a long way around to avoid it.

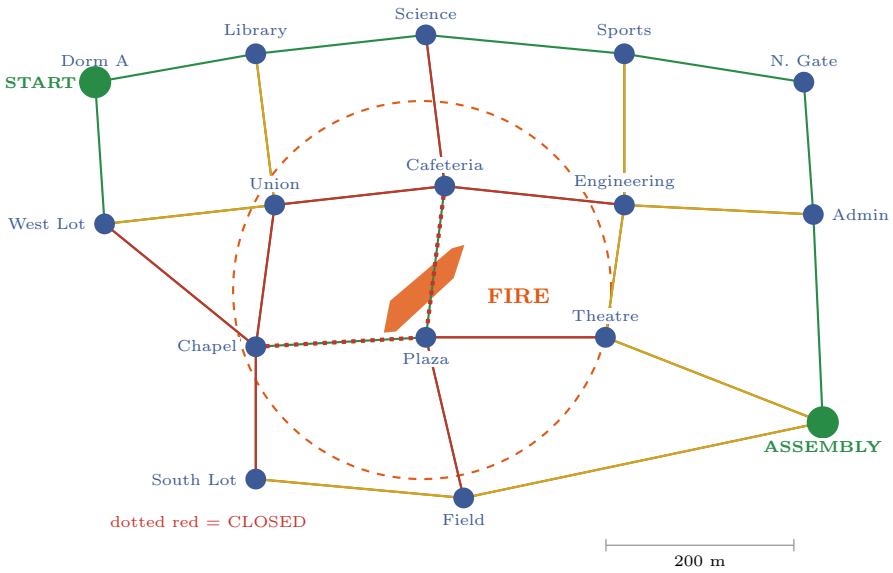


Figure 3: The same campus, coloured by danger. **Green** walkways are untouched by the fire; **amber** are mildly penalised; **red** are heavily penalised; and the two **dotted** walkways into Plaza are closed completely — nobody may use them.

These three numbers are a judgement call, not a law of nature. A fire chief might prefer a blocking distance of 50 m on a windy day, or a worst-danger of 20 for a chemical store. Change them in the program and the recommended routes change with them. What matters is that the *method* stays the same — and that whoever sets the numbers understands what they mean.

3 Step 3 — Finding the cheapest route with A⋆

We now have a graph where every walkway has an effective length. We want the route from Dorm A to the Assembly Point with the smallest total.

3.1 The obvious method, and why it is wasteful

You could list every possible route and add up each one. For a graph this size that is already millions of routes, and for a real city it is more routes than there are atoms in the Earth. We need something cleverer.

The classic method is **Dijkstra's algorithm**: start at the beginning and keep exploring outwards, always continuing from the cheapest place found so far. It works, and it always finds the true cheapest route. But it explores in *every* direction equally — including directly away from where you are trying to go.

3.2 A⋆: Dijkstra with a sense of direction

A⋆ adds one idea. When deciding which place to explore next, don't just ask "how much did it cost to get here?" — also ask "how much is probably left to go?"

For each place we track two numbers:

	Name	Meaning
g	cost so far	effective metres actually spent reaching this place
h	the hint	a *guess* at the effective metres still to go

and we always explore next whichever place has the smallest

$$\boxed{f \;=\; g \;+\; h} \tag{4}$$

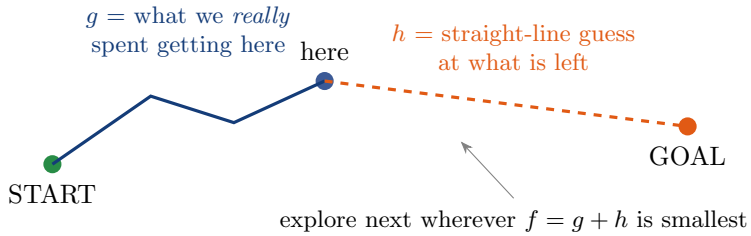


Figure 4: A* in one picture. g is real and already paid; h is a guess about the future. Adding them ranks places by how promising they look *overall*, so the search heads toward the goal instead of spreading out in all directions.

3.3 What makes a good hint?

We use the plain straight-line distance from a place to the goal — formula (1) again. But there is a rule the hint must obey, and it is the one genuinely subtle idea in this guide:

The hint must never be too big.

If the hint ever *overestimates* what is left, A* can be scared away from the route that was actually best, and it will confidently return a worse answer.

Our hint is safe, and here is the proof. Every danger factor is at least 1, so every walkway's effective length is at least its real length. That means the real remaining journey can never be *shorter* than the straight-line distance to the goal. So a straight-line hint can never be too big. ✓

An honest hint costs nothing and only makes the search faster. A dishonest one buys speed by giving wrong answers.

3.4 The algorithm, in words

1. Give the start $g = 0$. Every other place starts at $g = \infty$ ("no idea yet").
2. Put the start on a **to-visit list**.
3. Take off the to-visit list whichever place has the smallest $f = g + h$. Call it here.
4. If here is the goal — **done**.
5. Otherwise, look at each walkway leaving here. Skip any that are closed. For each neighbour, work out

   $$\text{new } g = g(\text{here}) + \text{effective length of the walkway.}$$

   If that beats the neighbour's current g, write it down, record that we arrived *from* here, and put the neighbour on the to-visit list.
6. Go back to step 3.

At the end, follow the "arrived from" notes backwards from the goal to the start — that is the route, in reverse.

On our campus, A* examined only **8 of the 16 places** before it had the answer. Dijkstra would have examined almost all of them. On a graph with a million places, that difference is the difference between an instant answer and a coffee break.

4 Step 4 — Suggesting a few routes, not just one

One recommendation is fragile. The fire spreads, a door is locked, a corridor fills with people. Emergency plans always offer *choices*.

4.1 How to get a second route

The idea is pleasingly blunt:

Close one walkway and ask again.

1. Find the best route with A*. Call it Route 1.
2. For each walkway *on* Route 1: pretend that one walkway is closed, and run A* again. Each run gives a candidate route that is forced to do something different.
3. Sort the candidates by cost, cheapest first.
4. Accept the cheapest candidate — but only if it is genuinely *different* (see below). Repeat until you have enough routes.

This is a simplified version of a method called **Yen's algorithm**, which is what real navigation apps use when they offer you three ways to drive somewhere.

4.2 Rejecting routes that are secretly the same

Without care, the "alternatives" come back nearly identical to the first — the same corridor with one block detoured around. That is useless in an emergency: if smoke closes that corridor, all three of your routes die at once.

So we measure the **overlap** between two routes:

$$\text{overlap} = \frac{\text{metres of walkway the two routes share}}{\text{total metres of the new route}} \tag{5}$$

and we reject any candidate that shares more than 60% of its walking with a route we have already accepted.

This rule really matters. Without it, our three suggested routes came out as *"north, then north again, then north a third time"* — all sharing their first four walkways. With it, the third suggestion is a completely separate corridor on the other side of campus, sharing **zero** walkways with the first.

5 The results

First, what the fire did to the walkways.

Of the 25 walkways, 8 are untouched, 15 are penalised, and **2 are closed completely** — both of the walkways leading into Fountain Plaza, which sits right beside the fire.

Now, the surprise.

If there were *no* fire, the best route would run along the south of campus:

Dorm A → West Lot → Chapel → Plaza → Theatre → Assembly, 974 m.

That route is now impossible — it goes straight through Plaza. **The fire has not merely lengthened the best route; it has moved it to the opposite side of campus.**

The three recommended routes.

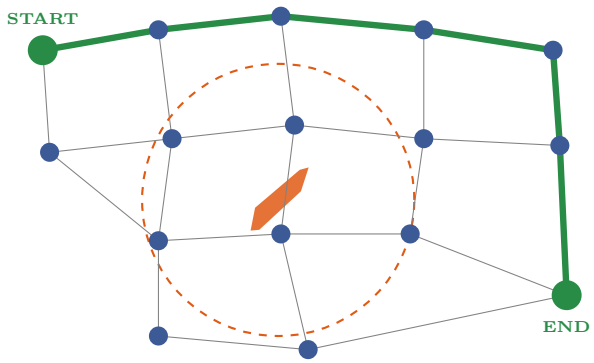
	Route	real metres	effective metres	worst danger
1	Dorm A → Library → Science → Sports → N. Gate → Admin → Assembly	1118	**1118**	×1.00
2	Dorm A → Library → Science → Sports → Engineering → Theatre → Assembly	1113	1768	×2.54
3	Dorm A → West Lot → Chapel → South Lot → Field → Assembly	1106	2437	×3.55

Read that table again — it is the whole point of this guide.

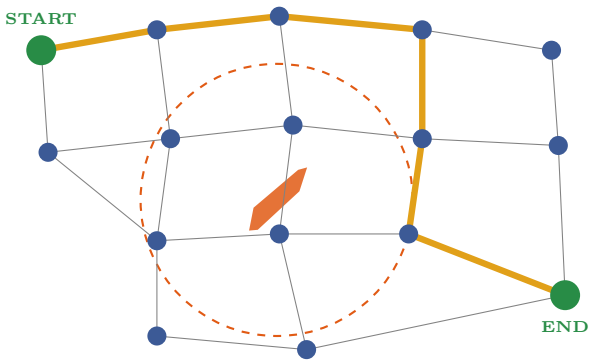
All three routes are about the same real walking distance. They differ by only 12 metres out of 1100 — less than 1%. In fact **Route 3 is the shortest of the three** in actual metres.

But their *effective* lengths differ by more than a factor of two, because Route 1 never comes near the fire while Route 3 spends much of its length in the danger zone. Route 1's worst danger factor is ×1.00: it is completely untouched by the fire.

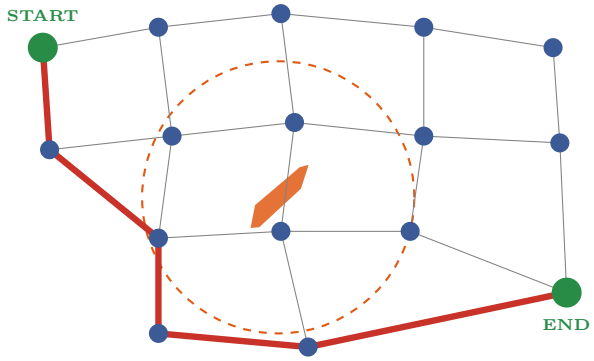
So the recommendation is: send everyone north on Route 1. Route 2 is a sensible fallback if the north gate is congested. Route 3 exists only because it uses entirely different walkways — if something goes wrong on the north side, it is the one that still works.



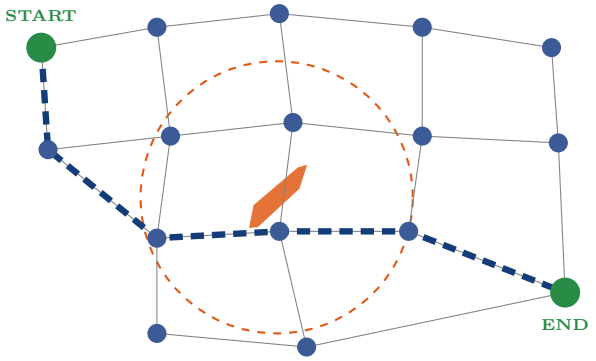
(a) **Route 1** — north. 1118 effective metres, never touched by the fire. **Recommended.**



(b) **Route 2** — north, then cutting south. 1768 effective metres. A reasonable fallback.



(c) **Route 3** — south. 2437 effective metres, but shares *no* walkway with Route 1.



(d) For comparison: the route we *would* have taken with no fire (974 m). It runs straight through the closed zone.

Figure 5: The three recommendations, and the route the fire took away from us. Notice that all four are about the same real length — the fire changes which one is *wise*, not which one is short.

Try it yourself #1.

1. The Union–Cafeteria walkway is 181.1 m long and passes 65.5 m from the fire. Use (3) to find its danger factor, then its effective length. *(The program says* 6.38 *and* 1155.7 *m*

— *do you agree?)*

2. The Theatre–Assembly walkway is 247.0 m long with $d = 172.3$ m. Find its effective length.
3. A walkway is 300 m long and passes 210 m from the fire. What is its effective length? (Careful — read (3) closely.)
4. If the fire chief raised the blocking distance from 25 m to 80 m, which extra walkways in the table on page 13 would close?

(Answers in Appendix B.)

6 Step 5 — When people start from everywhere

So far we have answered one question: *how do I get from Dorm A to the assembly point?* But in a real evacuation nobody is conveniently all in one building. There are people in the Library, the Cafeteria, the Theatre, the car parks. Every one of them needs an answer.

6.1 The obvious method, and the better one

You could run A* once for each building: sixteen searches for sixteen starting points. That works, and for a campus this size it is instant. But there is something much better — and the trick is worth knowing, because it scales to a whole city.

Turn the question around. Search *backwards* from the assembly point.

Start at the assembly point and let the search spread outwards across the whole campus, exactly as before but in reverse. Because a walkway costs the same in both directions, the cost the search assigns to a place *is* that place's cost to reach the assembly point.

One search answers the question for all sixteen starting points at once.

6.2 Why the hint has to go

Here is the one thing that genuinely changes. A*'s hint h is a guess at the distance to *the goal* — a single, specific goal. That is what lets it head in the right direction.

But now we no longer have one goal. Spreading out from the assembly point, we want to reach *everywhere*. There is no single direction to head in, so there is nothing for a hint to point at.

Set $h = 0$ and A* becomes **Dijkstra's algorithm** again — and that is exactly the right tool here. Far from being a step backwards, it is the correct choice: we *want* the search to spread in all directions, because we want an answer in all directions.

So: **A* for one start and one goal. Dijkstra backwards from the goal when the starts are everywhere.**

6.3 What the search leaves behind: a signpost at every place

As the search spreads, each place records *which neighbour it was reached from.* That neighbour is one step closer to the assembly point — so it is precisely where a person standing there should walk next.

Follow those signposts from any building and you arrive at the assembly point by the cheapest route. This is exactly how the green running-man signs in a building work: no sign shows you the whole route, each one just points to the next one.

A caution for one-way paths. Our walkways can be used in either direction, which is why searching backwards works so simply. If some route were one-way — a turnstile, a fire door that only opens outward — you would have to reverse those arrows before searching. The idea still works; you just have to be careful which way the arrows point.

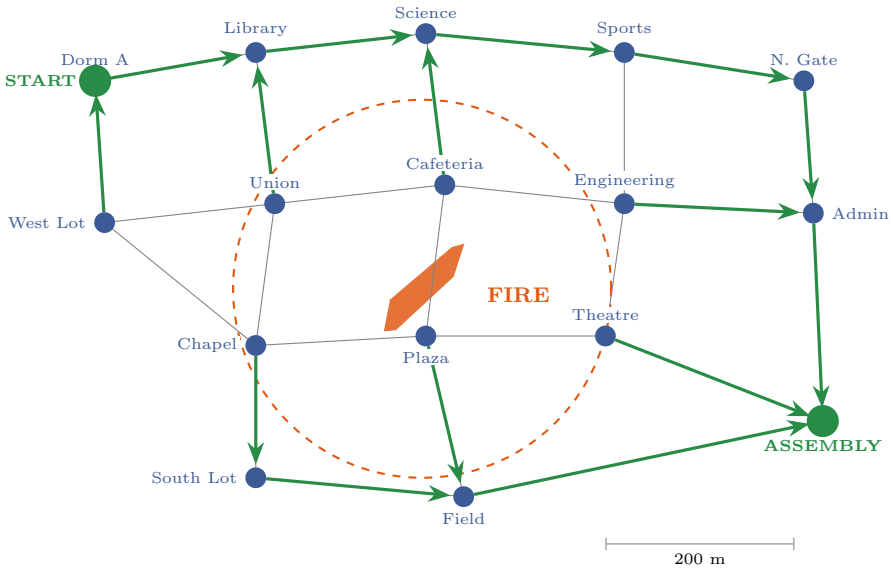


Figure 6: The **evacuation tree**, produced by one single backwards search. The arrow at each place is its signpost: walk that way. Follow the arrows from anywhere and you reach the assembly point by the cheapest route. Neither of the two closed walkways into Plaza is used. Only two arrows still run close to the fire — the ones leaving Plaza and the Cafeteria — and those are exactly the two places that have no safer option.

6.4 The signpost for every building

Straight from the program. "Effort" is the effective metres from that place to the assembly point — real distance already multiplied by danger, so it is the number that ranks how badly off you are.

You are at	effort	go to	and then onwards
Plaza	1931	Field	Field → Assembly
Cafeteria	1793	*Science*	Science → Sports → N. Gate → Admin → Assembly
Chapel	1555	South Lot	South Lot → Field → Assembly
Union	1373	Library	Library → Science → Sports → N. Gate → Admin → Assembly
West Lot	1268	*Dorm A*	Dorm A → Library → Science → Sports → N. Gate → Admin → Assembly
Dorm A	1118	Library	Library → Science → Sports → N. Gate → Admin → Assembly
South Lot	1058	Field	Field → Assembly
Library	945	Science	Science → Sports → N. Gate → Admin → Assembly
Science	764	Sports	Sports → N. Gate → Admin → Assembly
Engineering	626	Admin	Admin → Assembly
Sports	553	N. Gate	N. Gate → Admin → Assembly
Field	541	Assembly	(straight there)
Theatre	520	Assembly	(straight there)
N. Gate	361	Admin	Admin → Assembly
Admin	220	Assembly	(straight there)

The check that matters.

A* told us in Step 3 that Dorm A costs 1118 effective metres. This single backwards search says 1118 as well, by a completely different route through the calculation. Two different methods agreeing exactly is the best evidence that neither has a bug.

6.5 Two answers that look wrong, and are right

Look at the two rows printed in *italics* above.

From the Cafeteria, the sign says go *north* — but the assembly point is to the south-east.

The Cafeteria's direct route south runs into Fountain Plaza, and that walkway is closed by the fire. Its other southern option, through Engineering, hugs the fire and is penalised more than six times over. Going *away* from the assembly point, north to Science and around the top of campus, genuinely is the best thing to do.

West Lot is the same story: the sign points north to Dorm A, while the assembly point is south-east.

This is exactly the mistake a person would make on their own. Standing outside the Cafeteria, in smoke, the natural instinct is to set off toward the assembly point — and the natural instinct walks you straight into the fire. A signpost that has done the arithmetic does not have that instinct.

Plaza is the worst place to be — 1931 effective metres, nearly twice Dorm A's, even though Plaza is physically much closer to the assembly point. Both of its good exits are closed, so anyone there must go the long way round via the Field. If you were deciding where to send help first, that number tells you.

Try it yourself #2.

1. Start at the Union and follow the signposts in the table, place by place, until you reach the assembly point. How many walkways do you use?

2. The Union's effort is 1373 and the Library's is 945. What is the difference, and which walkway does it correspond to? Check it against the table in Appendix A.

3. The Theatre is 141 m *closer* to the assembly point than the Field is (247 m of walkway against 388 m) — and yet their efforts are almost the same, 520 against 541. Explain why. *(Hint: look up both walkways in Appendix A.)*

4. If you were the fire chief, which *two* buildings would you send help to first, and why?

(Answers in Appendix B.)

7 The C++ program

The file `campus_evacuation.cpp` does everything in this guide. Build and run it with:

```
g++ -std=c++17 -O2 -o campus_evacuation campus_evacuation.cpp
./campus_evacuation
```

It needs nothing but a C++ compiler — no libraries to install.

7.1 The three settings you are most likely to change

They sit at the top of the file so you do not have to hunt for them:

```
static const double BLOCK_RADIUS = 25.0;   // closer than this: walkway closed
static const double SAFE_RADIUS  = 200.0;  // further than this: fire ignored
static const double MAX_DANGER   = 8.0;    // penalty right at the block edge
static const int    N_ROUTES     = 3;      // how many routes to suggest
static const double MAX_OVERLAP  = 0.60;   // reject near-duplicate routes
```

7.2 Distance from a point to a walkway

This is formula (2) turned into code. Notice the `clamp` line — that is the "if t runs off the end, use the endpoint" rule:

```
static double pointToSegment(Point p, Point a, Point b) {
    double dx = b.x - a.x, dy = b.y - a.y;
    double len2 = dx * dx + dy * dy;
    if (len2 < 1e-12) return dist(p, a);      // a and b are the same point
    double t = ((p.x - a.x) * dx + (p.y - a.y) * dy) / len2;
    t = std::max(0.0, std::min(1.0, t));      // clamp onto the segment
    Point foot{a.x + t * dx, a.y + t * dy};
    return dist(p, foot);
}
```

7.3 The danger factor

Equation (3), almost word for word. `INF` is C++ for infinity, which is a tidy way to say "closed": a route through it would cost infinitely much, so the search will never pick it.

```
static double dangerFactor(double d) {
    if (d <= BLOCK_RADIUS) return INF;        // too close: closed
    if (d >= SAFE_RADIUS)  return 1.0;        // far enough: unaffected
    double howBad = (SAFE_RADIUS - d) / (SAFE_RADIUS - BLOCK_RADIUS); // 0..1
    return 1.0 + (MAX_DANGER - 1.0) * howBad;
}
```

7.4 A* itself

The heart of the program is shorter than you might expect. The `priority_queue` *is* the to-visit list: it automatically hands back whichever place has the smallest f.

```
auto h = [&](int v) { return dist(g.pos[v], g.pos[goal]); };  // the hint

std::priority_queue<Item, std::vector<Item>, std::greater<Item>> open;
gScore[start] = 0.0;
open.push({h(start), start});

while (!open.empty()) {
    auto [f, u] = open.top();  open.pop();
    if (done[u]) continue;
    done[u] = true;
    if (u == goal) break;                       // arrived

    for (const Edge& e : g.adj[u]) {
        if (banned.count(e.id)) continue;       // this walkway is closed
        if (e.cost == INF)      continue;       // the fire closed it
        double tentative = gScore[u] + e.cost;
        if (tentative < gScore[e.to] - 1e-9) {
            gScore[e.to]  = tentative;
            cameFrom[e.to] = u;                 // remember how we got here
            open.push({tentative + h(e.to), e.to});   //  f = g + h
        }
    }
}
```

The last line is equation (4) exactly: `tentative` is g, `h(e.to)` is h, and their sum is the f that decides what gets explored next.

One line turns A* back into Dijkstra. Change the hint to

```
auto h = [&](int v) { return 0.0; };
```

and the algorithm still gives the same, correct answer — it just explores more places to get there. That is a good experiment to run: A* needed 8 of the 16 places, and you can watch that number climb.

7.5 Reading the route back

A* never builds the route as it goes. It only leaves a trail of notes saying "I got to this place from that one". At the end we follow the notes backwards from the goal and then flip the list around:

```
for (int v = goal; v != -1; v = cameFrom[v]) out.path.push_back(v);
std::reverse(out.path.begin(), out.path.end());
```

A Every walkway, and what the fire did to it

Straight from the program. "To fire" is the closest the walkway gets to the fire patch; "effective" is real length × danger.

Walkway	metres	to fire (m)	danger	effective (m)
Dorm A – Library	172.6	291.2	1.00	172.6
Library – Science	181.1	225.9	1.00	181.1
Science – Sports	211.0	217.3	1.00	211.0
Sports – North Gate	192.4	263.6	1.00	192.4
Dorm A – West Lot	150.3	312.8	1.00	150.3
North Gate – Admin	140.4	370.6	1.00	140.4
Admin – Assembly	220.2	370.3	1.00	220.2
Field – Assembly	388.3	190.2	1.39	541.2
Sports – Engineering	160.0	174.4	2.03	324.1
Engineering – Admin	200.2	174.4	2.03	405.6
Theatre – Assembly	247.0	172.3	2.11	520.1
South Lot – Field	220.9	166.6	2.34	516.4
Engineering – Theatre	141.4	161.5	2.54	359.1
Library – Union	161.2	158.7	2.65	427.6
West Lot – Union	181.1	158.7	2.65	480.3
West Lot – Chapel	206.2	136.3	3.55	731.8
Chapel – South Lot	140.0	136.3	3.55	497.0
Union – Chapel	151.3	132.3	3.71	561.3
Science – Cafeteria	161.2	65.5	6.38	1028.9
Union – Cafeteria	181.1	65.5	6.38	1155.7
Cafeteria – Engineering	191.0	59.7	6.61	1263.5
Plaza – Theatre	190.0	26.0	7.96	1512.1
Plaza – Field	174.6	26.0	7.96	1389.8
Chapel – Plaza	180.3	7.4	**CLOSED**	
Cafeteria – Plaza	161.2	0.0	**CLOSED**	

Cafeteria–Plaza has a distance of exactly 0.0 because that walkway runs *through* the fire patch, not merely near it.

B Answers to "Try it yourself"

Set #1 — the danger factor.

1. $d = 65.5$ m is between 25 and 200, so

$$\text{danger} = 1 + 7 \times \frac{200 - 65.5}{175} = 1 + 7 \times \frac{134.5}{175} = 1 + 7 \times 0.7686 = \mathbf{6.38}.$$

   Effective length $= 181.1 \times 6.38 = \mathbf{1155}$ m. Agrees with the program.

2. $\text{danger} = 1+7\times\dfrac{200 - 172.3}{175} = 1+7\times0.1583 = 2.11$, so the effective length is $247.0\times2.11 = \mathbf{521}$ m. (The program prints 520.1, because it uses the unrounded distance.)

3. **Trick question: the effective length is just 300 m.** Because $210 \geq 200$, the third line of (3) applies and the danger factor is exactly 1. The fire is far enough away to be ignored entirely.

4. Looking at the "to fire" column for values under 80 m, five more walkways would close: **Science–Cafeteria** and **Union–Cafeteria** (65.5 m), **Cafeteria–Engineering** (59.7 m), and **Plaza–Theatre** and **Plaza–Field** (26.0 m each). That would cut Cafeteria off from everywhere except Science… which is itself only reachable from the north. The middle of campus would be sealed off completely.

Set #2 — everybody at once.

1. Union → Library → Science → Sports → N. Gate → Admin → Assembly. That is 7 places, so **6 walkways**.

2. $1373 - 945 = \mathbf{428}$, which is the effective length of the **Library–Union** walkway (427.6 m in Appendix A; the table above is rounded). This is the general pattern: *the difference between two neighbouring places' efforts is exactly the effective length of the walkway between them* — which is precisely what the search was adding up as it spread outwards.

3. Because effort is not distance — it is distance × danger.

$$\text{Theatre: } 247.0 \times 2.11 = 521, \qquad \text{Field: } 388.3 \times 1.39 = 541.$$

   The Theatre's walkway is much shorter but passes closer to the fire (172 m away, penalty 2.11), while the Field's is longer but safer (190 m away, penalty 1.39). The penalty very nearly cancels out the Theatre's head start. **Being closer is not the same as being better off.**

4. **Plaza (1931) and the Cafeteria (1793)** — the two highest efforts on the whole campus. Plaza especially: it sits right beside the fire and *both* of its good exits are closed, so despite being physically close to the assembly point it is the worst-off place on campus. Anyone there has the longest, most dangerous walk ahead of them.

C Words you now know

Word	What it means
graph	dots joined by lines; here, places joined by walkways
node, vertex	one of the dots — a place you can stand
edge	one of the lines — a walkway between two places
weight, cost	the number attached to an edge; ours is effective metres
effective length	real length × danger factor
Dijkstra's algorithm	finds the cheapest route by always expanding the cheapest place found so far
A* ("A-star")	Dijkstra plus a hint about which way the goal is
g	cost actually spent getting to a place
h	the heuristic: a guess at the cost still to go
$f = g + h$	how promising a place looks overall
heuristic	an educated guess used to steer a search
admissible	a heuristic that never overestimates — the safe kind
priority queue	a to-do list that always hands back the smallest item first
Yen's algorithm	the standard way to find the k best distinct routes
next hop, signpost	the one place to walk to next, recorded at every place
evacuation tree	all the signposts together; every arrow leads to the goal

D If you want to take this further

Genuine improvements you could make, roughly easiest first:

1. **Let the fire grow.** Re-run the cameras every minute, get a new fire patch, recompute the weights, and re-issue the routes. Nothing in the method needs to change — only the numbers.

2. **Add wind.** Smoke travels downwind, so the danger zone should be a stretched oval rather than a ring around the fire. Change one function — `distanceToFire` — and everything else follows.

3. **Add crowding.** A corridor with 500 people in it is slow. Make the danger factor depend on how many routes you have already sent along a walkway, and the system will spread people out by itself.

4. **Several assembly points.** Real campuses have more than one muster station. Give the backwards search of §6 *all* of them as starting points at once, and every building's signpost automatically points to whichever muster station is cheapest for it.

5. **Check it is still true.** Before publishing a route, confirm the point where the cameras say the fire is really does lie inside the patch you used — the consistency check from the camera guide.

The campus map, the fire patch and every number in this guide come from `campus_evacuation.cpp`. The fire itself was located in the companion guide, `fire_localization_2d.pdf`.